School of Computer Science

COMP5200M Scoping and Planning Document

UNIVERSITY OF LEEDS

Student Name: Sohan Kumar Reddy Gogula	
Programme of Study: M.Sc. Advanced Computer Science (Artificial Intelligence)	
Provisional Title of Project: An Intelligent Hybrid Approach to Machine Scheduling using Reinforcement Learning and Genetic Algorithms	
Name of External Company (if any)**:**	
Supervisor Name: **Steven Noble**	
Type of Project: **Research and software development project**	
***NOTE to student**: ensure you have discussed the content with the supervisor before submitting this document to Minerva. Submit an **electronic version** of this report in pdf via the appropriate link in Minerva; with filename of the format <surname><year>-SP (e.g. SMITH15-SP.pdf).*	
Signature of Student: **Sohan Kumar Reddy Gogula**	**Date: 20-03-2026**

Contents

1. Background Research for the project

1.1 Context

This project is motivated by a concrete industrial scheduling problem in textile dyeing. A dyeing facility operates multiple identical machines, each of which processes a sequence of colour batches (jobs). Each job consists of dyeing a batch of fabric or thread to a specified colour, with a known batch length (determining processing time) and a customer-imposed due date. When a machine transitions from one colour to the next, a period of waste is produced as residual dye is flushed from the system before the new colour runs cleanly. Critically, this transition cost is asymmetric: switching from a dark colour to a light colour (e.g., red to yellow) produces substantially more waste than the reverse, since residual dark dye contaminates the lighter shade for longer. This asymmetry gives rise to a rich, non-trivial sequencing problem. Production planners at such facilities must decide both how to assign batches across machines and in what order to run them on each machine, so as to meet customer due dates while minimising the total waste generated by colour transitions. This project directly addresses this industrial challenge using methods from the M.Sc. Advanced Computer Science (Artificial Intelligence) programme.

1.2 Problem statement

Traditional heuristic methods struggle to maintain efficiency in dynamic scheduling environments. The core problem is the optimal scheduling of colour-dyeing batches across m identical parallel machines, where transitioning between colour types incurs asymmetric, sequence-dependent setup costs (waste) and setup times (machine downtime). Traditional scheduling heuristics such as Shortest Processing Time (SPT) treat all transitions as equivalent and cannot exploit the structure of the colour-transition cost matrix. Kim and Lee (1998) demonstrate that classical heuristics consistently underperform on heterogeneous machine scheduling problems with sequence-dependent costs, while Chien and Lan (2021) show that agent-based approaches integrating evolutionary and learning methods yield significant efficiency gains over rule-based dispatching in similarly structured problems. This results in unnecessarily high waste and frequent due date violations when colour sequences are poorly ordered. The research hypothesis is that a hybrid GA–DRL system, by learning to navigate the asymmetric cost matrix intelligently, can substantially outperform classical heuristics on both waste minimisation and tardiness reduction.

Formal Problem Definition

The scheduling problem is formally defined as a Parallel Machine Scheduling Problem with Sequence-Dependent Setup Costs (PMSP-SDSC). Given a set of *n* jobs to be processed on *m* parallel machines, the goal is to find an assignment and ordering of

jobs across machines that minimises both total makespan and the sum of sequence-dependent transition (setup) costs, subject to due date constraints.

Sets and Indices

$J = \{1, 2, ..., n\}$ — the set of all jobs, indexed by i or j.

$M = \{1, 2, ..., m\}$ — the set of all machines ($m \geq 2$), indexed by k.

Job Parameters

p_i — processing time of job i, i.e., the time units required to complete job i once it begins processing on any machine. This corresponds to the run time for a given colour batch, determined by the batch length and dye absorption rate.

d_i — due date of job i. The deadline by which job i must be completed. Lateness is defined as $L_i = C_i - d_i$, and tardiness (penalised lateness) as $T_i = max(0, L_i)$.

r_i — release time (or ready time) of job i. The earliest time at which job i becomes available for scheduling. May be set to 0 for all jobs in the initial model.

w_i — weight (priority) of job i. A positive scalar representing job importance, used to form the weighted tardiness objective $\sum w_i \cdot T_i$. Set to 1 for all jobs in the unweighted baseline variant.

Setup (Transition) Cost Parameters

$s(i, j)$ — sequence-dependent setup cost incurred when job j immediately follows job i on the same machine. This is represented as an $n \times n$ asymmetric cost matrix S, where $S[i][j] \neq S[j][i]$ in general. For example, transitioning from red to yellow produces more waste than yellow to red, since residual red dye contaminates the lighter shade for longer. The diagonal $S[i][i] = 0$ for all i (no cost when the same job type repeats consecutively).

$st(i, j)$ — setup time incurred when transitioning from job i to job j on the same machine. This is the downtime (in time units) during which the machine is unavailable. Distinct from the cost $s(i,j)$, though both may be modelled from the same matrix in simplified variants.

Decision Variables

σ^k — the ordered sequence (permutation) of jobs assigned to machine k. A complete solution is the collection $\Sigma = \{\sigma_1, \sigma_2, ..., \sigma^k\}$, where every job in J appears in exactly one σ^k.

C_i — completion time of job i. Derived from the sequence σ^k and computed recursively: the completion time of the l-th job in a sequence equals the previous job's completion time plus the setup time plus the current job's processing time.

Objective Function

The project optimises a composite objective that balances two competing concerns. The primary objective is to minimise total weighted tardiness across all jobs: $f_1(\Sigma) = \sum_i w_i \cdot T_i$. The secondary objective is to minimise the total sequence-dependent setup cost across all machines: $f_2(\Sigma) = \sum^k \sum_l s(\sigma^k(l), \sigma^k(l+1))$. The combined fitness function used by the GA is: $F(\Sigma) = \alpha \cdot f_1(\Sigma) + (1 - \alpha) \cdot f_2(\Sigma)$, where $\alpha \in [0, 1]$ is a tunable weighting parameter. Makespan $C_{max} = max_i(C_i)$ is tracked as a secondary diagnostic metric.

Constraints

Each job must be assigned to exactly one machine (no job splitting). Each machine processes at most one job at a time (non-preemptive scheduling). A job may not begin processing before its release time r_i. Setup time $st(i, j)$ must be fully elapsed before job j can begin. All jobs must be scheduled (no job may be dropped to reduce cost). Due date violations are penalised via the tardiness function rather than treated as hard constraints, enabling the optimiser to make trade-off decisions.

1.3 Possible solution

The proposed solution is an intelligent hybrid optimisation system that combines the broad exploratory power of a Genetic Algorithm (GA) with the adaptive, policy-driven decision-making of a Deep Reinforcement Learning (DRL) agent. Depending on the speed of progress, the DRL agent (likely PPO or DQN) will act as a 'hyper-heuristic' to control the GA parameters. The fundamental building blocks for this solution will be implemented in Python, leveraging established libraries including Gymnasium for environment simulation and DEAP for evolutionary computation.

Solution Representation. Each GA chromosome encodes a complete schedule as a partitioned permutation: a ordered list of all n jobs partitioned into m contiguous segments, one per machine. The position of a job within its segment defines its processing order on that machine, and hence determines all completion times C_i and the total transition cost $F(\Sigma)$. Crossover and mutation operators are designed to preserve the partition structure, ensuring every offspring represents a valid feasible schedule with no job duplicated or omitted.

Reinforcement Learning Formulation. The DRL agent operates at the level of the GA rather than individual schedules. The **state** is a vector summarising the current generation's population: best fitness, mean fitness, population diversity (measured by average pairwise Hamming distance between chromosomes), and a stagnation counter tracking generations without improvement. The **action** is a discrete choice from a set of operator configurations, such as selecting between mutation strategies

(e.g. swap, inversion, insertion) or adjusting the mutation rate. The **reward** is the relative improvement in best fitness over the subsequent k generations, encouraging the agent to select operators that actively drive progress rather than merely maintaining the current best solution.

1.4 How to demonstrate the quality of the solution

Solution quality will be evaluated through direct comparison against established benchmarks such as Shortest Processing Time (SPT) which is a standard genetic algorithm baseline and classical heuristic method. Meanwhile, performance metrics are to be measured using metrics like make span reduction and computational cost. Providing objective, reproducible evidence of improvement over each reference method.

The results are to be communicated through visual aids like Gantt charts and performance graphs that shall clearly illustrate the efficiency and allow intuitive comparison between techniques. Together, these quantitative metrics and visualisations constitute a rigorous and transparent basis for validating the practical viability of the proposed hybrid approach.

Testing and Evaluation Methodology

Benchmark Instances. As no standard dyeing-specific benchmark dataset exists in the literature, synthetic problem instances will be procedurally generated. Each instance specifies a set of n jobs (colour batches), each with a randomly drawn batch length (processing time), due date, and colour type drawn from a fixed palette. The asymmetric colour-transition cost matrix S will be constructed to reflect realistic dye-waste behaviour: transitions from darker to lighter colours carry higher costs. Multiple instance sizes will be tested: n = 10 (small, tractable instances used for manual verification and debugging), n = 20 (medium, representative of a realistic single-day dyeing schedule at a small facility), and n = 50 (large enough to stress-test GA convergence and demonstrate scalability). Machine counts of m = 2 and m = 3 are chosen as m = 2 is the minimum for a meaningful parallel machine problem, while m = 3 introduces a more complex job-assignment space without making the search space computationally intractable. All experiments will be run on a standard laptop CPU; since GA runs are independent across repetitions, the 30 runs per configuration will be parallelised using Python's multiprocessing library.

Baseline Comparisons. Each algorithm will be evaluated on identical problem instances. The baselines are: (1) Shortest Processing Time (SPT) heuristic, which ignores transition costs entirely; (2) a nearest-neighbour greedy heuristic, which always selects the next job with the lowest setup cost from the current job; and (3) a standard GA without the DRL hyper-heuristic layer. The hybrid GA–DRL system is expected to outperform all three on the composite objective F(Σ).

Statistical Rigour. Since both the GA and DRL agent are stochastic, all experiments will be repeated across at least 30 independent runs per algorithm per instance. Results will be reported as mean ± standard deviation. Where appropriate, a Wilcoxon signed-rank test will be used to assess whether observed improvements are statistically significant rather than artefacts of random variation.

Convergence Analysis. Fitness curves will be plotted across generations for the GA and hybrid system to verify that the DRL agent actively improves convergence speed and solution quality relative to the plain GA. Gantt chart visualisations of representative schedules will illustrate how the hybrid system groups compatible colour batches to minimise transition waste compared to baseline orderings.

2. Scope for this project

2.1 Aim

The aim of this project is to develop a hybrid optimisation system that minimises total make-span and sequence-dependent setup costs (e.g., "waste" during transitions) for industrial machine scheduling.

2.2 Objectives

- Conduct a targeted literature review focused on the application of Reinforcement Learning within meta-heuristics for Job Shop Scheduling problems with sequence-dependent setup costs.
- Implement a mathematical model of a multi-job, multi-machine scheduling environment incorporating asymmetric transition costs, treating the derivation of physical cost parameters as a secondary concern.
- Develop a standard Genetic Algorithm (GA) as a performance baseline, with clearly defined benchmarks for minimizing lateness and setup waste, against which subsequent improvements will be measured.
- Design and implement a Deep Reinforcement Learning (DRL) hyper-heuristic agent using PPO or DQN that dynamically adjusts the GA's mutation rates or selects local search operators during execution.
- Comparatively, evaluate the proposed hybrid system against both the GA baseline and established heuristics such as SPT, using makespan and computational cost as the primary performance metrics.

2.3 Deliverables

- Completion of the final dissertation report, which will detail the entire process from the initial methodology and system design to the critical evaluation of hybrid system effectiveness.

- Comprehensive technical documentation regarding the technical system architecture, including the rationale behind the cost matrices and the reward functions used in the RL agent.
- A set of visualized results, including Gantt charts and performance graphs, to effectively present the benefits in efficiency resulting from the proposed approach in comparison with a baseline implementation.

3. Project schedule

3.1 Methodology

The project will follow an iterative, modular software development lifecycle adapted for machine learning research. The three core components which are the simulation environment, the baseline GA, and the DRL agent will be developed, tested, and validated independently before integration. This modular approach ensures that each algorithm is mathematically sound and behaviourally correct in isolation, preventing errors from compounding during the integration phase.

3.2 Tasks, milestones and timeline

A Gantt chart will be utilized to track the project phases and ensure milestones are met on time .

- Milestone 1 (Weeks 20–22, Mar–Apr 2026): Complete the targeted literature review and implement the mathematical model of the scheduling environment. Draft Chapter 2.
- Milestone 2 (Weeks 25–28, May–Jun 2026): Develop, test, and benchmark the standard Genetic Algorithm baseline against SPT and other classical heuristics.
- Milestone 3 (Weeks 29–S2, Jun–Jul 2026): Design and implement the DRL hyper-heuristic agent and integrate it with the GA environment. Present progress at the mid-project assessor meeting (Week S2).
- Milestone 4 (Weeks S3–S7, Jul–Aug 2026): Conduct comparative evaluations, generate all visualisations, and finalise the dissertation report for submission by 31 August 2026.

3.3 Risk assessment (if appropriate)

Identified Risk: The DRL agent may fail to converge on a stable optimal policy, or may demand excessive computational resources during training and evaluation.

Mitigation Strategy: Training will begin with deliberately simplified reward functions and small-scale job/machine configurations to enable rapid iteration and effective debugging. Complexity will be scaled incrementally only once stable convergence has been confirmed at each stage.

References

Alipour, M.M., Razavi, S.N., Feizi Derakhshi, M.R. and Balafar, M.A. (2018) 'A hybrid algorithm using a genetic algorithm and multiagent reinforcement learning heuristic to solve the traveling salesman problem', *Neural Computing and Applications*, 30(10), pp. 2935–2951. Available at: https://doi.org/10.1007/s00521-017-2880-4

Chien, C. and Lan, Y. (2021) 'Agent-based approach integrating deep reinforcement learning and hybrid genetic algorithm for dynamic scheduling for Industry 3.5 smart production', *Computers & Industrial Engineering*, 162, p. 107782. Available at: https://doi.org/10.1016/j.cie.2021.107782

Kim, G.H. and Lee, C.S.G. (1998) 'Genetic reinforcement learning approach to the heterogeneous machine scheduling problem', *IEEE Transactions on Robotics and Automation*, 14(6), pp. 879–893. Available at: https://doi.org/10.1109/70.736772

Yanamandram Kuppuraju, S., Sankaran, P. and Patil, S. (2025) 'Hybrid task scheduling using genetic algorithms and machine learning for improved cloud efficiency', *International Journal for Multidisciplinary Research*. Available at: https://www.semanticscholar.org/paper/CorpusID:277211285

Appendix A. How ethical issues are addressed

As this computational project relies entirely on synthetic mathematical models and standard open-source test data to simulate industrial machines, no ethical issues involving human participants or sensitive data are involved.